Capitolul 10
Algoritmi de clasificare

Cuprins

- Ce este învățarea supervizată
- Evaluare clasificatori
- Arbori de decizie: ID3 și C4.5
- Clasificatori bayesieni (Naïve Bayes)
- Mașini cu vectori suport (Support vector machines)
- K-nearest neighbors
- Metode asambliste: Bagging, Boosting, Random Forest

Obiective

- Invățarea supervizată este un subdomeniu foarte studiat al Data Mining.
- In acest caz se construiește un nou model din experiențe trecute (din date).

Definiții

- Învățarea supravegheată include:
  - Clasificare: rezultatele sunt valori discrete (obiectiv: identificarea apartenenței la grup / clasă).
  - Regresie: rezultatele sunt valori continue sau ordonate (obiectiv: estimarea sau prezicerea unui răspuns).
- In acest capitol ne concentram pe clasificare

Clasificarea

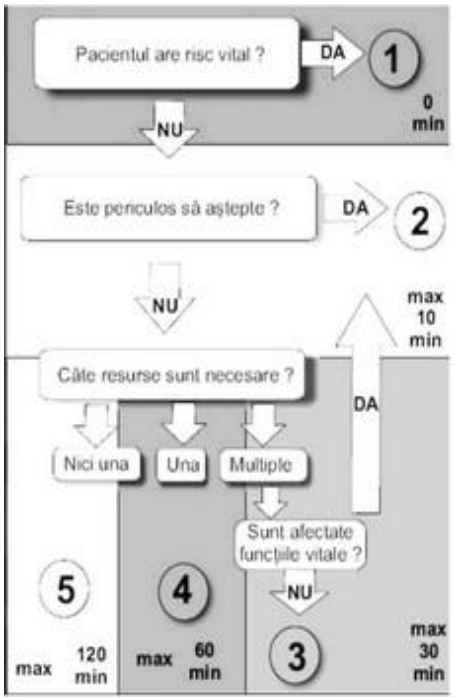
Intrare:

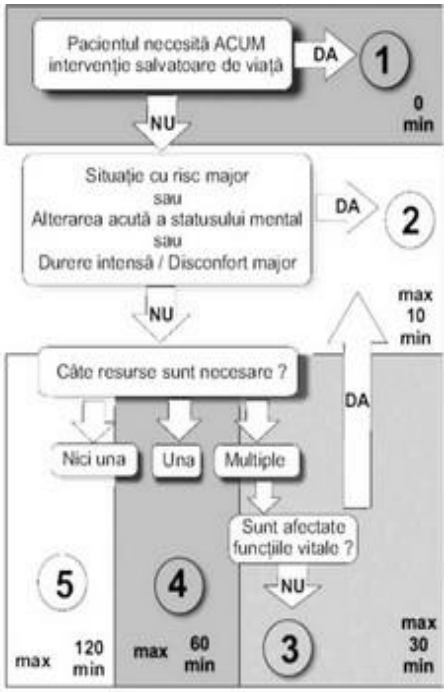
- Un set de **k** clase C = {c_1, c_2,…, c_k}
- Un set de n articole etichetate D = {(d_1, c_{i1}), (d_2, c_{i2}), ..., (d_n c_{in})}. Articolele sunt d1,…, dn, fiecare articol dj fiind etichetat cu o clasa cj din C. D se numește **mulțime/set de antrenare**.
- Pentru calibrarea unor algoritmi, este necesara o mulțime de validare. Aceasta conține de asemenea elemente etichetate care nu sunt incluse în setul de antrenare.

Ieşire:

- Un model sau o metodă pentru clasificarea articolelor noi. Mulțimea de articole noi care vor fi clasificate folosind modelul / metoda se numește **mulțime/set de test**.

Exemplu. Model: arbore de decizie





- Rezultatul pentru exemplul de test:

Felix	**Yes**	**Yes**	**No**	**No**	**No**	**Yes**	**?????**

este C0

Formatul datelor de intrare

- În majoritatea cazurilor, setul de antrenament (precum și setul de validare și setul de testare) pot fi reprezentate de un tabel având o coloană pentru fiecare atribut al lui X, iar ultima coloană conține eticheta clasei.
- Un exemplu foarte folosit este Play-tenis în care sunt folosite condițiile meteorologice pentru a decide dacă jucătorii pot sau nu începe un nou joc.
- Acest set de date va fi utilizat și în acest curs.

Setul Play tennis

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Arbori de decizie

- Arbori de decizie: un exemplu este arborele de decizie UPU așa cum este descris în exemplul precedent.
- Într-un arbore de decizie nodurile interne conțin decizii bazate pe valorile atributelor exemplelor (atribute ale argumentului x_i) și fiecare frunză y_i este o clasă din C.
- ID3 și C4.5 sunt doi algoritmi cunoscuți pentru construirea arborilor de decizie și sunt prezentați în acest curs.

Clasificatori bayesieni

- Clasificare bayesiană (Naïve Bayes): pentru fiecare exemplu x_i, metoda calculează probabilitatea pentru fiecare clasă y_j din C.
- Clasificarea se face alegând cea mai probabilă clasă pentru fiecare exemplu.
- Cuvântul naiv este folosit deoarece se fac unele presupuneri simplificatoare.

Maşini cu vectori suport

- Maşini cu vectori suport: această metodă este utilizată pentru clasificarea binară.
- Exemplele sunt clasificate în doar două clase: fie pozitive, fie negative.
- Este o metodă foarte eficientă şi poate fi folosită recursiv pentru a construi un clasificator cu mai mult de două clase.

KNN

- Cei mai apropiați K vecini (K-nearest neighbors): o metodă foarte simplă, dar puternică pentru clasificarea exemplelor bazate pe etichetele vecinilor.

Metode asambliste

- Metode asambliste: Random Forest, Bagging și Boosting.
- În aceste cazuri, mai mult de un clasificator este construit, iar clasificarea finală se face prin agregarea rezultatelor.
- De exemplu, un clasificator de tip Random Forest constă în mai mulți arbori de decizie, iar clasa de ieșire este valoarea modala (cea mai frecventa) a claselor returnate de arborii individuali.

Cuprins

- Ce este învățarea supervizată
- Evaluare clasificatori
- Arbori de decizie: ID3 și C4.5
- Clasificatori bayesieni (Naïve Bayes)
- Mașini cu vectori suport (Support vector machines)
- K-nearest neighbors
- Metode asambliste: Bagging, Boosting, Random Forest

Acuratețea și rata de eroare

- Pentru estimarea eficienței unui clasificator pot fi utilizate mai multe măsuri:
- Acuratețea (sau acuratețea predictivă) este proporția exemplelor de test corect clasificate de model (sau metodă):

$$\text{Acuratețea} = \frac{\text{Numărul de exemple de test corect clasificate}}{\text{Numărul total de exemple de test}}$$

- Rata de eroare este proporția exemplelor de test clasificate incorect:

$$\text{Rata de eroare} = 1 - \text{Acuratețea}$$

Alte măsuri

- În unele cazuri exemple sunt clasificate în doar două clase (numite pozitive și negative). Atunci pot fi definite și alte măsuri.
- Să considerăm matricea de confuzie care conține numărul de exemple clasificate corect și incorect (exemple pozitive, precum și exemple negative):

	Classified as Positive	Classified as Negative
Actual positive	TP = True Positive	FN = False Negative
Actual negative	FP = False Positive	TN = True Negative

Alte măsuri

- TP = numărul de clasificări corecte pentru exemple pozitive.
- TN = numărul de clasificări corecte pentru exemple negative.
- FP = numărul de clasificări incorecte pentru exemple negative.
- FN = numărul de clasificări incorecte pentru exemple pozitive.

- **Precizia** este proporția exemplelor pozitive clasificate corect din mulțimea exemplelor clasificate ca pozitive:

Precizie = TP / (TP + FP)

Alte măsuri

- **Senzitivitatea** (Recall, sensitivity) este proporția de exemple pozitive corect clasificate în mulțimea exemplelor pozitive (sau rata de recunoaștere a exemplelor pozitive):

  Senzitivitatea = TP / (TP + FN)

- **Specificitatea** (Specificity) este rata de recunoaștere a exemplelor negative:

  Specificitatea = TN / (TN + FP)

Alte măsuri

- Formula acurateței poate fi rescrisă astfel:

$$Accuracy = \frac{Recall * Pos}{Pos + Neg} + \frac{Specificity * Neg}{Pos + Neg}$$

Unde Pos și Neg sunt numărul total de exemple pozitive și negative

Alte măsuri

- Precizia şi senzitivitatea sunt utilizate de obicei împreuna: pentru unele mulțimi de test folosind doar una din ele poate duce la o judecată incorectă.
- Exemplu: Un set conține 100 de exemple pozitive şi 100 de exemple negative iar clasificatorul produce următorul rezultat:

	Clasificate Pozitive	Clasificate negative
Pozitive	**30**	**70**
Negative	**0**	**100**

Alte măsuri

- Atunci precizia p = 100% dar senzitivitatea r = 30%. Putem combina precizia și senzitivitatea obținând scorul F_1 (**F_1-score)** care este media lor armonica:

$$F1\text{–}score = \frac{2}{\frac{1}{p} + \frac{1}{r}} = \frac{2pr}{p + r}$$

- Pentru exemplul anterior F_1-score = 46%; în general F_1-score este mai aproape de valoarea mai mică a celor doua.

Metode de evaluare

- Metodele de evaluare utilizează un set de date **D** cu exemple etichetate.
- Acest set este divizat în mai multe subseturi și aceste subseturi devin seturi de antrenare / test / validare.

- Notă: Evaluarea se referă la metoda / algoritmul de construire a clasificatorului.

Metoda Holdout

- În acest caz, setul de date D se împarte în două: un set de antrenare şi un set de test.
- Setul de test se mai numeşte **set de reţinere** - ***holdout set*** (de aici numele metodei).
- Clasificatorul obţinut folosind setul de antrenare este utilizat pentru clasificarea exemplelor din setul de test.
- Deoarece aceste exemple sunt, de asemenea, etichetate, se pot calcula acurateţea, precizia, senzitivitatea şi alte măsuri şi pe baza lor este evaluat clasificatorul.

Validarea încrucișată

Există mai multe versiuni de validare încrucișată:

1. Validare încrucișată folosind k subseturi - ***k-fold cross validation***.

   Setul de date D este împărțit în k subseturi disjuncte cu aceeași dimensiune.

   Pentru fiecare subsetse construiește un clasificator folosind acel subset ca set de test și reuniunea subseturilor rămase ca set de antrenare.

   În acest fel se obțin k valori pentru acurateță (una pentru fiecare clasificator). Media acestor valori este acurateța finală. Valoarea obișnuită pentru k este 10.

2. Validare încrucișată folosind 2 subseturi - ***2-fold cross validation***. Pentru k = 2, metoda de mai sus are avantajul de a folosi seturi mari atât pentru antrenament, cât și pentru testare.

Validarea încrucișată

3. Validare încrucișată stratificată - ***Stratified cross validation***. Este o variație de validare încrucișată având k subseturi. Fiecare subset are aceeași distribuție a etichetelor.

- De exemplu, pentru exemple pozitive și negative, fiecare subset conține aproximativ aceeași proporție de exemple pozitive și aceeași proporție de exemple negative.

4. Validarea încrucișată având subseturi de un element - ***Leave one out cross validation*** . Când D conține doar un număr mic de exemple, o validare încrucișată specială poate fi folosită: fiecare exemplu devine set de test și toate celelalte exemple sunt setul de antrenare.

- Acuratețea pentru fiecare clasificator este fie 100%, fie 0%. Media tuturor acestor valori este acuratețea finală.

Metoda bootstrap

- Metoda bootstrap: face parte din metodele cu eșantionare și constă în obținerea setului de antrenare din setul de date original prin eșantionare cu înlocuire (eșantionul nu este înlăturat din set).
- Cazurile care nu sunt alese în setul de antrenament sunt utilizate ca set de testare.
- De exemplu, dacă D are 1000 de exemple etichetate, alegând la întâmplare un exemplu de 1000 de ori ne oferă setul de antrenare. În acesta unele exemple sunt de mai multe ori.

Metoda bootstrap

- Statistic, 63,2% din exemplele din D sunt alese pentru setul de antrenare iar 36,8% nu. Aceste 36,8% devin setul de test.
- După construirea unui clasificator şi rularea acestuia pe setul de test este determinată acurateţea şi astfel poate fi evaluată metoda de construire a clasificatorului în funcţie de valoarea obţinută.
- Mai multe despre această metodă şi alte tehnici de evaluare pot fi găsite în [Sanderson 08].

De ce 63,2%?

- Din Data Mining: Concepts and Techniques*: să presupunem că avem un set de date cu **d** exemple. Fiecare exemplu are o probabilitate de **1/d** de a fi selectat, deci probabilitatea de a nu fi ales este **(1-1/d)**.
- Cum alegem de **d** ori, probabilitatea ca un exemplu să nu fie ales în tot acest timp este $\mathbf{(1-1/d)^d}$.
- Dacă d este mare, probabilitatea se apropie de $\mathbf{e^{-1}} = 0.368$. Astfel, 36,8% din exemple nu vor fi selectate pentru setul de antrenare şi vor ajunge în setul de test iar restul de 63,2% vor forma setul de antrenare.

* Jiawei Han, Micheline Kamber, Data Mining: Concepts and Techniques, Second Edition, 2006, Morgan Kaufman, pagina 365.

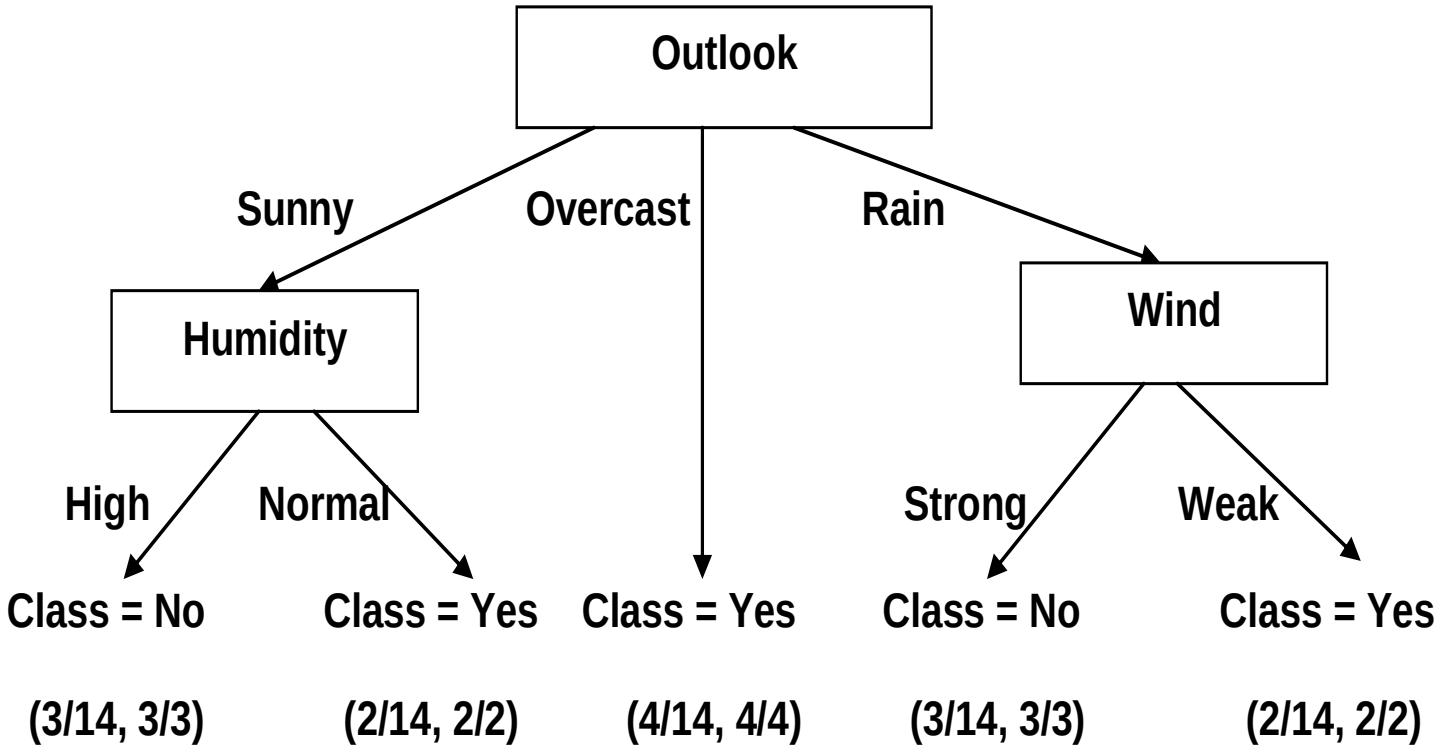
Cuprins

- Ce este învățarea supervizată
- Evaluare clasificatori
- Arbori de decizie: ID3 și C4.5
- Clasificatori bayesieni (Naïve Bayes)
- Mașini cu vectori suport (Support vector machines)
- K-nearest neighbors
- Metode asambliste: Bagging, Boosting, Random Forest

Ce este un arbore de decizie?

- Un mod comun de a reprezenta un model sau un algoritm de clasificare este un arbore de decizie. Având un set de antrenare D şi un set A de atribute ale exemplelor, fiecare exemplu etichetat din D are forma: ($a_1 = v_1$, $a_2 = v_2$, …, $a_n = v_n$). Pe baza acestor atribute se poate construi un arbore de decizie având:
  - Nodurile interne sunt atribute (fără nicio cale conţinând de două ori acelaşi atribut).
  - Ramurile se referă la valorile discrete (una sau mai multe) sau intervale pentru aceste atribute. Uneori pot fi utilizate condiţii mai complexe pentru ramificare.
  - Frunzele sunt etichetate cu clase. Pentru fiecare frunză se poate calcula un suport şi o încredere: suportul este proporţia de exemple care se potrivesc cu calea de la rădăcină la frunza respectivă, iar încrederea este acurateţea clasificării pentru exemplele care se potrivesc cu acea cale. La trecerea de la arbori de decizie la reguli, fiecare regulă are acelaşi suport şi aceeaşi încredere ca frunza din care provine.
  - Orice exemplu se potriveşte cu o singură cale a arborelui (deci o singură frunză = clasă).

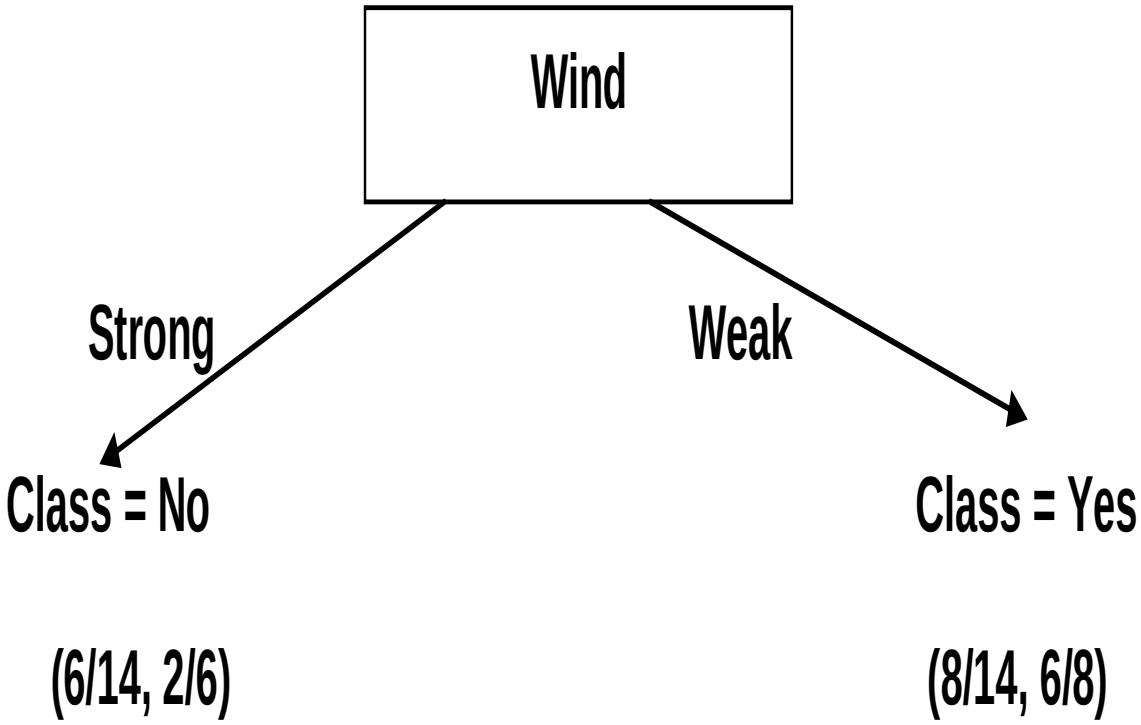
Exemplu


Outlook
Sunny
Overcast
Rain
Humidity
Wind
High
Normal
Strong
Weak
Class = No
Class = Yes
Class = Yes
Class = No
Class = Yes
(3/14, 3/3)
(2/14, 2/2)
(4/14, 4/4)
(3/14, 3/3)
(2/14, 2/2)

Exemplu

- Numerele de pe ultima linie sunt suportul şi încrederea asociate fiecărei frunze.
- Pentru acelaşi set de date pot fi creaţi mai mulţi arbori de decizie.
- De exemplu un alt arbore de decizie pentru acelaşi set de date se află în figura următoare (cu valori mai mici ale încrederii decât arborele anterior):

Decision trees



ID3

- ID3 (Iterativ Dichotomiser 3) este un algoritm pentru construirea arborilor de decizie introdus de Ross Quinlan în 1986 (vezi [Quinlan 86]).
- Algoritmul construieşte arborele de decizie într-o manieră top-down alegând la fiecare nod atributul „cel mai bun” pentru ramificare:
- Mai întâi este ales un atribut rădăcină, construind o ramură separată pentru fiecare valoare diferită a atributului.

ID3

- Setul de instruire este de asemenea împărțit, fiecare ramură moștenind exemplele care se potrivesc cu valorile atributelor ramurii respective.
- Procesul se repetă pentru fiecare descendent până când toate exemplele au aceeași clasă (în acest caz nodul devine o frunză etichetată cu acea clasă) sau toate atributele au fost utilizate (nodul devine, de asemenea, o frunză etichetată cu valoarea modală - clasa majoritară).
- Un atribut nu poate fi ales de două ori pe aceeași cale; din momentul în care a fost ales pentru un nod, nu va mai fi niciodată testat pentru descendenții acelui nod.

Cel mai bun atribut

- Esența ID3 este modul în care este descoperit atributul „cel mai bun”. Algoritmul folosește teoria informației încercând să crească puritatea seturilor de date de la nodul tată la descendenți.
- Să luăm în considerare un set de date:

  $D = \{e_1, e_2, \ldots, e_m\}$

  cu exemple etichetate cu clase din

  $C = \{c_1, c_2, \ldots, c_n\}$.

  Atributele exemplelor sunt $A_1, A_2, \ldots, A_p$.
- Atunci Entropia lui D poate fi calculată astfel:

Entropia

$$entropy(D) = -\sum_{i=1}^{n} \Pr(c_i) \log_2 \Pr(c_i)$$

- Dacă atributul A_k având **r** valori distincte este considerat pentru ramificare, acesta va partiționa D în subseturile disjuncte D_1, D_2, ..., D_r.
- Entropia combinată a acestor subseturi, calculată ca medie ponderată a acestor entropii este:

$$entropy(D, A_k) = \sum_{i=1}^{r} \frac{count(D_i)}{count(D)} * entropy(D_i)$$

- Toate probabilitățile implicate în ecuațiile de mai sus sunt determinate prin numărare!

Câștigul informațional

- Deoarece puritatea seturilor de date crește, entropia (D) este mai mare decât entropia (D, A_k). Diferența dintre ele se numește câștig informațional:

$$\boldsymbol{Gain(D, A_k) = entropy(D) - entropy(D, A_k)}$$

- Cel mai bun atribut este cel având câștigul informațional maxim.

Exemplu

- Pentru setul de date Play tenis există patru atribute posibile pentru rădăcina arborelui decizional: Aspect, Temperatură, Umiditate și Vânt.
- Entropia întregului set de date și valorile ponderate pentru împărțirea utilizării celor patru atribute sunt:

$$entropy(D) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.94$$

Pentru fiecare atribut

$$entropy(D, Humidity) = \frac{7}{14}\left(-\frac{3}{7}\log_2\frac{3}{7} - \frac{4}{7}\log_2\frac{4}{7}\right) + \frac{7}{14}\left(-\frac{6}{7}\log_2\frac{6}{7} - \frac{6}{7}\log_2\frac{6}{7}\right) = 0.79$$

- La fel:

$$entropy(D, Wind) = 0.89$$

$$entropy(D, Temperature) = 0.91$$

$$entropy(D, Outlook) = 0.69$$

Cel mai bun atribut: Outlook

- Următorul tabel conține valorile pentru entropie și câștig.
- Cel mai bun atribut pentru nodul rădăcină este Aspect (Outlook), cu câștigul maxim de 0,25:

Attribute	entropy	gain
Humidity	0.79	0.15
Wind	0.89	0.05
Temperature	0.91	0.03
Outlook	0.69	0.25

Discuție ID3

1. Deoarece este un algoritm greedy ID3 produce un optim local.
2. Atributele cu multe valori distincte duc la un câștig mai mare. Pentru rezolvarea acestei probleme câștigul poate fi înlocuit cu raportul de câștig (**gain-ratio**):

$$gain\text{–}ratio(D, A_k) = \frac{gain(D, A_k)}{entropy(D, A_k)}$$

Discuție ID3

3. Uneori (când doar câteva exemple sunt asociate cu frunzele) arborele se mulează pe datele de instruire (overfitting) și nu funcționează bine la alte exemple de test.

- Pentru a evita overfittingul arborele poate fi simplificat prin reducere (tăiere - pruning):
  - Pre-pruning : creșterea este oprită înainte de sfârșitul normal. Frunzele nu sunt 100% pure și sunt etichetate cu clasa majoritară (valoarea modală).
  - Post-pruning: după rularea algoritmului, unii subarbori sunt înlocuiți cu frunze. De asemenea, în acest caz, etichetele sunt valori modale pentru exemplele din setul de antrenare care se potrivesc. Tehnica post-pruning este mai bună, deoarece în pre-pruning este greu de estimat când se trebuie sa ne oprim.

Discuție ID3

4. Unele atribute A pot fi continue. Valorile pentru A pot fi împărțite în două intervale:

   A ≤ t și **A > t**.

- Valoarea lui t poate fi selectată după cum urmează:
  - Sortează exemple după A
  - Alege ca și valoare candidat media a două valori consecutive în care clasa se schimbă.
  - Pentru fiecare valoare candidat de la pasul anterior calculează câștigul dacă partiționarea se face folosind acea valoare. Candidatul cu câștig maxim este ales pentru partajare.

Discuție ID3

- În acest fel, atributul continuu este înlocuit cu unul discret (două valori, una pentru fiecare interval).
- Acest atribut concurează cu atributele rămase pentru „cel mai bun” atribut.
- Procesul se repetă pentru fiecare nod, deoarece valoarea partiționării se poate schimba de la un nod la altul.

Discuție ID3

5. Costul atributelor: unele atribute sunt mai scumpe decât altele (măsurate nu numai în bani).

- Este mai bine ca atributele cu costuri mai mici să fie mai aproape de rădăcină decât alte atribute.
- De exemplu, pentru o unitate de urgență, este mai bine să testăm pulsul și temperatura mai întâi și numai atunci când este necesar să efectuăm o biopsie.
- Acest lucru se poate face prin ponderarea câștigului cu costul:

$$weighted\text{–}gain(D, A_k) = \frac{gain(D, A_k)}{cost(A_k)}$$

C4.5

- C4.5 este versiunea îmbunătățită a ID3 și a fost dezvoltată de același Ross Quinlan (precum și C5.0). Câteva caracteristici:
  - Sunt permise atribute numerice (continue)
  - Tratează cazul valorilor lipsă
  - Utilizează post-pruning pentru a face față la date cu zgomot.
- Cele mai importante îmbunătățiri de la ID3 sunt:

1. Atributele sunt alese pe baza raportului de câștig și nu pur și simplu a câștigului informațional.

C4.5

2. Folosește post-pruning pentru a reduce dimensiunea arborelui. Tăierea se face numai dacă reduce eroarea estimată. Există două metode de tăiere:
   - Înlocuire sub-arbore: Un sub-arbore este înlocuit cu o frunză, dar fiecare sub-arbore este considerat numai după toți sub-arborii săi. Aceasta este o abordare de jos în sus.
   - Ridicare sub-arbore: Un nod este ridicat şi înlocuiește un nod superior. Dar în acest caz, unele exemple trebuie reasignate. Această metodă este considerată mai puțin importantă şi mai lentă decât prima.

Cuprins

- Ce este învățarea supervizată
- Evaluare clasificatori
- Arbori de decizie: ID3 și C4.5
- Clasificatori bayesieni (Naïve Bayes)
- Mașini cu vectori suport (Support vector machines)
- K-nearest neighbors
- Metode asambliste: Bagging, Boosting, Random Forest

Naïve Bayes: Generalități

- Această abordare este una probabilistică.
- Algoritmii bazați pe teorema lui Bayes calculează pentru fiecare exemplu de test nu o singură clasă, ci probabilitatea pentru fiecare clasă din C (setul de clase).
- Dacă setul de date are k atribute , A_1, A_2, …, A_k, obiectivul este să calculăm pentru fiecare clasă $c \in C = \{c_1, c_2, \ldots, c_n\}$ probabilitatea ca exemplul de test (a_1, a_2, …, a_k) să aparțină clasei c:

$$\mathbf{Pr(Class = c \mid A_1 = a_1, ..., A_k = a_k)}$$

- Dacă este necesară clasificarea, clasa cu cea mai mare probabilitate poate fi atribuită acelui exemplu.

Teorema lui Bayes

- Thomas Bayes (1701 - 1761) a fost un cleric presbiterian și un matematician englez.
- Teorema numită după el poate fi exprimată astfel:

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

unde P (A | B) înseamnă probabilitatea lui A fiind dat B.

Exemplu

- Studenții din EC004 sunt 70% de la C5 și 30% opționali.
- 20% dintre studenți sunt plasați în primele 3 rânduri, dar pentru opționali acest procent este de 40%.
- Când rectorul intră în sala și se așează undeva în primele 3 rânduri, lângă un student, caare este probabilitatea ca vecinul său să fie din categoria opționali? ”

Soluție

$$Pr(Opt) = 0.3$$

$$Pr(3 \text{ rânduri} \mid Opt) = 0.4)$$

$$Pr(3 \text{ rânduri}) = 0.2$$

- Deci:

$$Pr(Opt \mid 3 \text{ rânduri}) = \frac{(Pr(3 \text{ rânduri} \mid Opt) * Pr(Opt))}{Pr(3 \text{ rânduri})} =$$

$$= 0.4 * 0.3 / 0.2 = 0.6 \text{ sau } 60\%$$

Construirea modelului

- Obiectivul este de a calcula:

$$\Pr(\text{Class} = c \mid A_1 = a_1, \ldots, A_k = a_k).$$

- Aplicând teorema lui Bayes:

$$\Pr(C = c_j \mid A_1 = a_1, \ldots, A_k = a_k) =$$

$$\frac{\Pr(A_1 = a_1, \ldots, A_k = a_k \mid C = c_j) * \Pr(C = c_j)}{\Pr(A_1 = a_1, \ldots, A_k = a_k)} =$$

$$\frac{\Pr(A_1 = a_1, \ldots, A_k = a_k \mid C = c_j) * \Pr(C = c_j)}{\sum_{x=1}^{n} \Pr(A_1 = a_1, \ldots, A_k = a_k \mid C = c_x) \Pr(C = c_x)}$$

Construirea modelului

- Facem următoarea presupunere: „toate atributele sunt condițional independente dându-se clasa C = c_j” atunci:

$$\Pr(A_1 = a_1, \ldots, A_k = a_k \mid C = c_j) = \prod_{i=1}^{k} \Pr(A_i = a_i \mid C = c_j)$$

- Din cauza acestei presupunere metoda este numită „naivă”.
- Presupunerea nu este valabilă în toate situațiile,.
- Practica arată însă că rezultatele obținute folosind această presupunere simplificatoare sunt suficient de bune în majoritatea cazurilor.

Construirea modelului

- În final, înlocuind expresia anterioară în formulă obținem:

$$\Pr(C = c_j \mid A_1 = a_1, \ldots, A_k = a_k) =$$

$$\frac{\Pr(C = c_j) * \prod_{i=1}^{k} \Pr(A_i = a_i \mid C = c_j)}{\sum_{x=1}^{n} \Pr(C = c_x) \prod_{i=1}^{k} \Pr(A_i = a_i \mid C = c_x)}$$

- Toate probabilitățile din expresia de mai sus pot fi obținute prin numărare!

Construirea modelului

- Când este necesară numai clasificarea numitorul expresiei de mai sus poate fi ignorat (este același pentru toți c_j), iar clasa de etichetare se obține prin maximizarea numărătorului:

$$C = argmax_{cj} \Pr(c_j) * \prod_{i=1}^{k} \Pr(A_i = a_i | C = c_j)$$

Exemplu

- Fie versiunea simplificată a tabelului PlayTennis:

Outlook	Wind	Play Tennis
Overcast	Weak	Yes
Overcast	Strong	Yes
Overcast	Absent	No
Sunny	Weak	Yes
Sunny	Strong	No
Rain	Strong	No
Rain	Weak	No
Rain	Absent	Yes

Exemplu

Pr(Yes) = 4/8
Pr(Overcast | C = Yes) = 2/4
Pr(Overcast | C = No) = 1/4
Pr(Sunny | C = Yes) = 1/4
Pr(Sunny | C = No) = 1/4
Pr(Rain | C = Yes) = 1/4
Pr(Rain | C = No) = 2/4

Pr(No) = 4/8
Pr(Weak | C = Yes) = 2/4
Pr(Weak | C = No) = 1/4
Pr(Strong| C = Yes) = 1/4
Pr(Strong| C = No) = 2/4
Pr(Absent| C = Yes) = 1/4
Pr(Absent| C = No) = 1/4

- Daca exemplul de test este:

Sunny	**Absent**	**???**

Exemplu

For C = Yes

$$\Pr(Yes) * \Pr(Sunny \mid Yes) * \Pr(Absent \mid Yes) = \frac{4}{8} * \frac{1}{4} * \frac{1}{4} = \frac{1}{32}$$

For C = No

$$\Pr(No) * \Pr(Sunny \mid No) * \Pr(Absent \mid No) = \frac{4}{8} * \frac{1}{4} * \frac{1}{4} = \frac{1}{32}$$

- Rezultatul este că ambele probabilități au aceeași valoare și clasa poate fi oricare dintre ele.

Caz special: împărțire la 0

- Sometimes a class does not occur with a specific attribute value.
- In that case one term Pr(Ai = ai | C = cj) is zero, so the above expression for probabilities of each class evaluates to 0/0.
- For avoiding this situation, the expression:

$$\Pr(A_i = a_i | C = c_j) = \frac{a}{b}$$

trebuie modificată.

(a = numărul de exemple de antrenare cu Ai = ai și C = cj și b = numărul de exemple de antrenare cu C = cj)

Caz special: împărțire la 0

- Expresia modificată este:

$$\Pr(A_i = a_i | C = c_j) = \frac{a + s}{b + s * r}$$

unde:

- s = 1 / Număr de exemple din setul de antrenare
- r = Numărul valorilor distincte pentru Ai

- În acest caz toți termenii expresiei sunt mai mari decât zero.

Caz special: valori

Valori non-categorice sau valori absente:

- Toate atributele care nu sunt categorice trebuie discretizate (înlocuite cu unele categorice).
- De asemenea, dacă unele atribute au valori lipsă, aceste valori sunt ignorate.

Cuprins

- Ce este învățarea supervizată
- Evaluare clasificatori
- Arbori de decizie: ID3 și C4.5
- Clasificatori bayesieni (Naïve Bayes)
- Mașini cu vectori suport (Support vector machines)
- K-nearest neighbors
- Metode asambliste: Bagging, Boosting, Random Forest

SVM: Generalități

- Este prezentată doar ideea generală a metodei de clasificare Support Vector Machines (SVM).
- SVM-urile sunt descrise în detaliu în multe documentații și cărți, de exemplu [Liu 11] sau [Han, Kamber 06].
- Metoda a fost descoperită în Uniunea Sovietică în anii ’70 de Vladimir Vapnik și a fost dezvoltată în SUA după ce Vapnik s-a alăturat AT&T Bell Labs la începutul anilor ’90 (a se vedea [Cortes, Vapnik 95]).

SVM: Model

- Considerăm o mulțime de antrenare:

$$D = \{(X_1, y_1), (X_2, y_2), ..., (X_k, y_k)\}$$

unde:

- $X_i = (x_1, x_2, ..., x_n)$ este un vector din R^n (toate componentele lui x_i sunt numere reale)
- y_i este eticheta de clasă, cu $y_i \in \{-1, +1\}$. Dacă X_i este etichetat cu +1 aparține clasei pozitive, altfel aparține clasei negative (-1).

SVM: Model

- Un posibil clasificator este o funcție liniară:

$$f(X) = \langle w \cdot X \rangle + b$$

Astfel încât:

$$y_i = \begin{cases} 1 \; if \; \langle w \cdot X \rangle + b \geq 0 \\ -1 \; if \; \langle w \cdot X \rangle + b < 0 \end{cases}$$

unde:

- **w** este un vector de ponderi (weight vector),
- $\langle w \cdot X \rangle$ este produsul scalar intre vectorii **w** și **X**,
- **b** este un număr real, și
- **w** și **b** pot fi scalați, după cum se va vedea.

SVM: Model

- Semnificația lui **f** este că hiperplanul

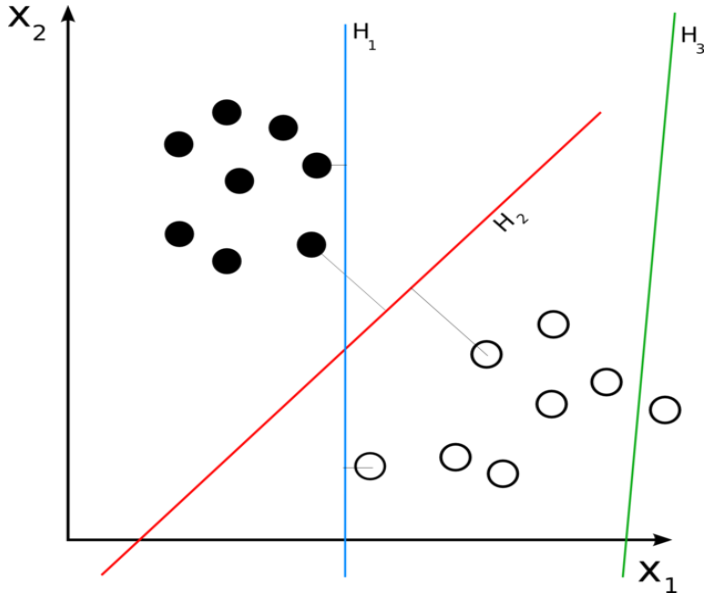
$$< w \cdot X> + b = 0$$

separă punctele setului de antrenare D în două:

- o jumătate din spațiu conține valorile pozitive și
- cealaltă jumătate valorile negative din D (precum hiperplanurile H1 și H2 din figura următoare).

- Toate exemplele de test pot fi acum clasificate folosind f: valoarea f dă eticheta pentru exemplu.

Figura 1

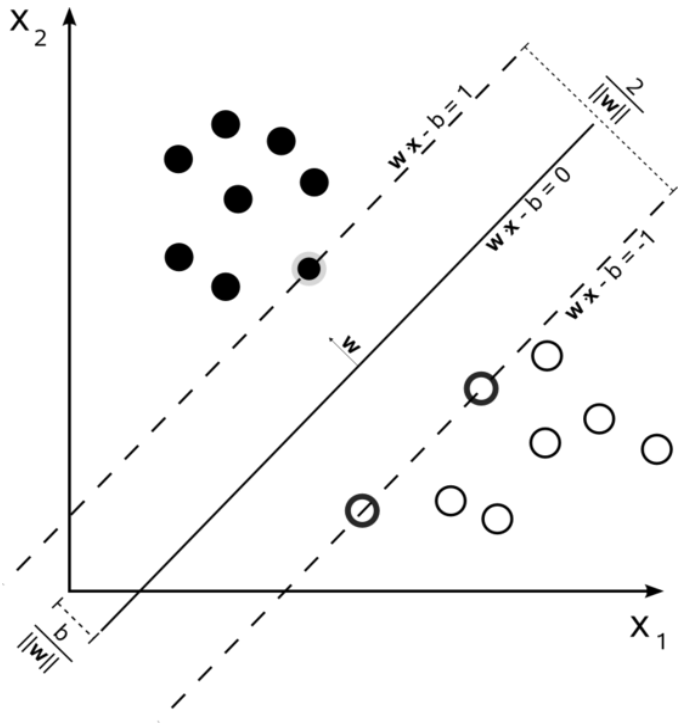


- Source: Wikipedia

Cel mai bun hiperplan

- SVM încearcă să găsească „cel mai bun” hyperplan de acea formă.
- Teoria arată că cel mai bun plan este cel care maximizează așa-numita “margine” (distanța ortogonală minimă între un punct pozitiv și negativ din setul de antrenament - a se vedea figura următoare pentru un exemplu.

Figura 2



- Sursa: Wikipedia

Separarea non-liniară

- În multe situații nu există un hiperplan care separă exemplele pozitive de cele negative.
- În astfel de cazuri este posibilă maparea punctelor din setul de antrenare (exemplele din D) într-un alt spațiu dimensional superior.
- Aici punctele pot fi liniar separabile.
- Funcția de mapare primește exemple (vectori) din spațiul de intrare X și le mapează în așa-numitul spațiu caracteristic (feature space) F:

$$\phi : X \rightarrow F$$

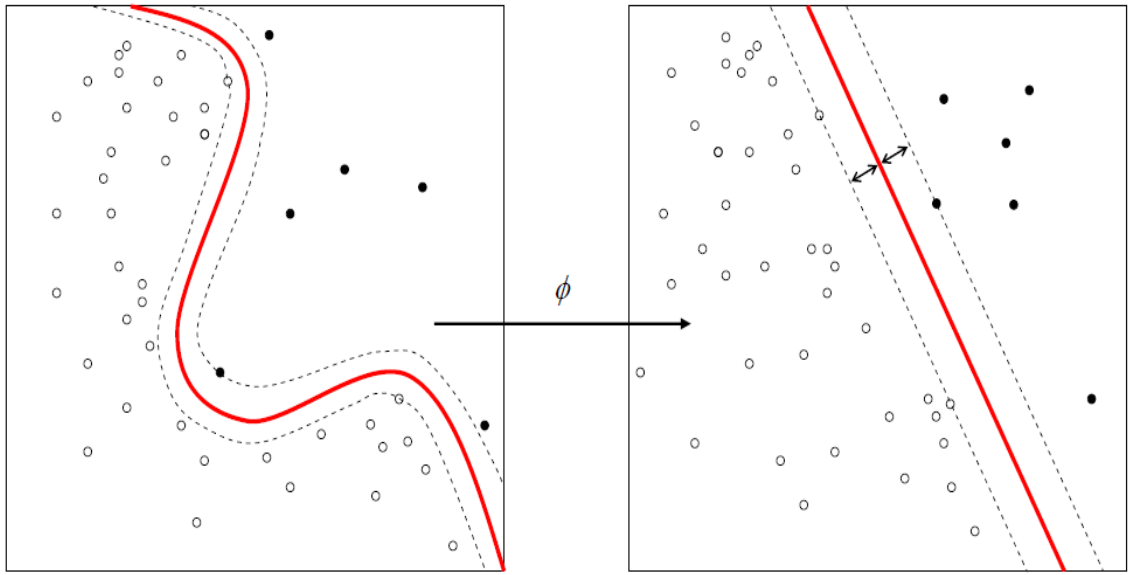
Separarea non-liniară

- Fiecare punct X din D este mapat în $\phi(X)$. După maparea tuturor punctelor avem alt set de antrenare conținând vectori din F și nu din X, cu dim(F) $\geq$ n = dim(X):

$$D = \{(\phi(X_1), y_1), (\phi(X_2), y_2), ..., (\phi(X_k), y_k)\}$$

- Pentru un ϕ corespunzător aceste puncte sunt liniar separabile.

Figura 3



- Sursa: Wikipedia

Funcţii kernel

- Dar cum putem găsi această funcţie de mapare?
- În soluţionarea problemei de optimizare pentru găsirea hiperplanului de separare liniară în noul spaţiu de funcţii F toţi termenii care conţin exemple de instruire sunt doar de forma $\phi(X_i) \cdot \phi(X_j)$.
- Înlocuind acest produs scalar cu o funcţie atât în Xi cât şi în Xj, nevoia de a găsi ϕ dispare. O astfel de funcţie se numeşte **funcţie kernel**:

$$K(X_i, X_j) = \phi(X_i) \cdot \phi(X_j)$$

- Pentru a găsi hiperplanul de separare în F trebuie doar să înlocuim toate produsele scalare cu funcţia de nucleu aleasă şi apoi să continuăm cu problema de optimizare ca în cazul separabil.

Funcții kernel

Unele dintre cele mai utilizate funcții kernel sunt:

- Kernel liniar:

$$K(X, Y) = <X \cdot Y> + b$$

- Kernel polinomial:

$$K(X, Y) = (a * <X \cdot Y> + b)^p$$

- Kernel sigmoid:

$$K(X, Y) = \tanh(a * <X \cdot Y> + b)$$

Discuție SVM

- SVM se folosește când atributele au valori reale
  - Când în datele de instruire există atribute categorice, este necesară o conversie la valori reale.
- Când sunt necesare mai mult de două clase, SVM poate fi utilizat recursiv.
  - Prima utilizare separă o clasă; a doua utilizare separă clasa a doua și așa mai departe. Pentru N clase sunt necesare rulaje N-1.
- SVM sunt o metodă foarte bună în clasificarea datelor hiper-dimensionale.

Cuprins

- Ce este învățarea supervizată
- Evaluare clasificatori
- Arbori de decizie: ID3 și C4.5
- Clasificatori bayesieni (Naïve Bayes)
- Mașini cu vectori suport (Support vector machines)
- K-nearest neighbors
- Metode asambliste: Bagging, Boosting, Random Forest

kNN

- Metoda “Cei mai apropiaţi k vecinii” - K-nearest neighbors (kNN) nu produce un model, ci este o metodă simplă pentru determinarea clasei unui exemplu bazat pe etichetele vecinilor săi aparţinând setului de antrenare.
- Pentru rularea algoritmului este necesară o funcţie de distanţă pentru calcularea distanţei de la exemplul de test până la exemplele din setul de antrenare.
- O funcţie f (x, y) poate fi utilizată ca funcţie de distanţă dacă sunt îndeplinite patru condiţii:

1. $f(x, y) \geq 0$
2. $f(x, x) = 0$
3. $f(x, y) = f(y, x)$
4. $f(x, y) \leq f(x, z) + f(z, y)$.

Algoritm

Intrare:

- Un set de date **D** care conține exemple etichetate (setul de antrenare)
- O funcție de distanță **f** pentru măsurarea distanței între două exemple
- Un întreg **k** - parametru care spune câți vecini sunt luați în considerare
- Un exemplu de test **t**

Ieşire:

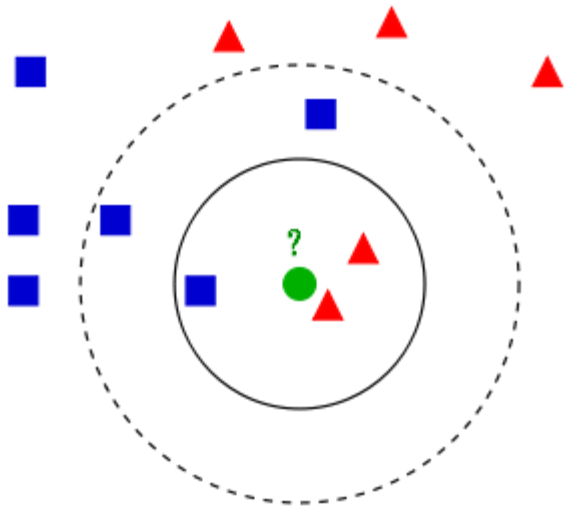
- Clasa lui **t**

Metodă:

- Folosiți **f** pentru a calcula distanța dintre **t** şi fiecare punct din D
- Selectați cele mai apropiate puncte **k**
- Alocați clasa majoritară din setul de **k** vecini apropiați.

Exemplu

- K = 3 → Roșu
- K = 5 → Albastru



- kNN este foarte sensibil la valoarea parametrului **k**.
- Cea mai bună valoare pentru **k** poate fi găsită, de exemplu, prin validare încrucișată.

Cuprins

- Ce este învățarea supervizată
- Evaluare clasificatori
- Arbori de decizie: ID3 și C4.5
- Clasificatori bayesieni (Naïve Bayes)
- Mașini cu vectori suport (Support vector machines)
- K-nearest neighbors
- Metode asambliste: Bagging, Boosting, Random Forest

Metode asambliste

- Metodele asambliste combină mai mulți clasificatori “slabi” pentru a obține unul mai bun (mai “puternic”).
- Clasificatorii combinați sunt similari (utilizează aceeași metodă de învățare), dar seturile de antrenare și ponderile exemplelor din ele sunt diferite.

Bagging

- Numele Bagging vine de la **B**ootstrap **Agg**regat**ing**.
- Așa cum am văzut mai devreme metoda bootstrap face parte din metodele de eșantionare și constă în obținerea unui set de antrenare din datele etichetate inițiale prin eșantionare cu înlocuire.

Exemplu

Setul original	a	b	c	d	e	f
Set antrenare 1	**a**	**b**	**b**	**c**	**e**	**f**
Set antrenare 2	**b**	**b**	**c**	**c**	**d**	**e**
Set antrenare 3	**a**	**b**	**c**	**c**	**d**	**f**

Bagging

Tehnica Bagging constă în:

- Pornind de la setul de date original, construiți **n** seturi de date de antrenare prin eșantionare cu înlocuire (eșantioane bootstrap)
- Pentru fiecare set de date de antrenare, construiți un clasificator folosind același algoritm de învățare (se obțin n clasificatori slabi).
- Clasificatorul final se obține prin combinarea rezultatelor clasificatorilor slabi (prin vot, de exemplu).
- Baggingul ajută la îmbunătățirea preciziei pentru algoritmi instabili de învățare: arbori de decizie, rețele neuronale.
- Nu ajută în cazul kNN, Naïve Bayesian sau CAR.

Boosting

- Tehnica Boosting constă în construirea unei secvențe de clasificatori slabi și adăugarea lor în structura clasificatorului final puternic.
- Clasificatorii slabi sunt ponderați în funcție de acuratețe.
- De asemenea, datele sunt reevaluate după ce construim fiecare clasificator slab, astfel încât exemplele clasificate incorect câștigă în greutate.
- Rezultatul este că următorii clasificatori slabi din secvență se concentrează mai mult pe exemplele pe care le-au ratat clasificatorii slabi precedenți.

Random forest

- Tehnica Random forest este un clasificator asamblist format dintr-un set de arbori de decizie Clasificatorul final produce valoarea modală a claselor de ieşire de fiecare arbore.
- Algoritmul este următorul:

1. Alege T - numărul de arbori de construit.
2. Alegeţi m - numărul de variabile utilizate pentru ramificare la fiecare nod. m << M, unde M este numărul de variabile de intrare.
3. Construim T arbori. Pentru fiecare:

- Construim un set de antrenare prin eşantionare cu înlocuire. Din el se creşte un arbore de decizie.
- La fiecare nod, selectăm m variabile la întâmplare şi le folosiţm pentru a găsi cele mai bune ramificări.
- Creste arborele la maxim. Nu există tăieri.

4. Se prezic rezultatele agregând predicţiilor arborilor (de ex. vot majoritar pentru clasificare, medie pentru regresie).

Referințe

- [Liu 11] Bing Liu, 2011. Web Data Mining, Exploring Hyperlinks, Contents, and Usage Data, Second Edition, Springer, chapter 3.
- [Han, Kamber 06] Jiawei Han, Micheline Kamber, Data Mining: Concepts and Techniques, Second Edition, Morgan Kaufmann Publishers, 2006
- [Cortes, Vapnik 95] Cortes, Corinna; and Vapnik, Vladimir N.; "Support-Vector Networks", Machine Learning, 20, 1995. http://www.springerlink.com/content/k238jx04hm87j80g/
- [Wikipedia] Wikipedia, the free encyclopedia, en.wikipedia.org
- [Sanderson 08] Robert Sanderson, Data mining course notes, Dept. of Computer Science, University of Liverpool 2008, Classification: Evaluation http://www.csc.liv.ac.uk/~azaroth/courses/current/comp527/lectures/comp527-13.pdf
- [Quinlan 86] Quinlan, J. R. 1986. Induction of Decision Trees. Mach. Learn. 1, 1 (Mar. 1986), 81-106, http://www.cs.nyu.edu/~roweis/csc2515-2006/readings/quinlan.pdf